See also

Show Attributes and Methods [described]

Object Paths in SimTalk

Names

Names [conventions]

All objects and local variables in *Plant Simulation* are identified by their names or their paths. Certain limitations apply when choosing a new name. You cannot, for example, use the names of the keywords as the name of an object or a local variable. Self-explanatory names, such as `.building1` for a *Frame* or `forklift` for a *Transporter*, make working with and maintaining a model easier for yourself and for the colleagues maintaining the simulation model.

Remarks

Naming conventions are described under **User-defined Names in Methods**.

> **Note**
>
> *SimTalk* normally does not distinguish between upper- and lower-casing for the names of *methods*, *attributes*, and *read-only attributes*, which you type into the source code of your **Method** objects.

See also

Name [general description]

Keywords

Predefined Names

User-defined Names in Methods

Plant Simulation automatically assigns valid names to new objects you create. Within *control methods* you yourself have to select names for variables.

Remarks

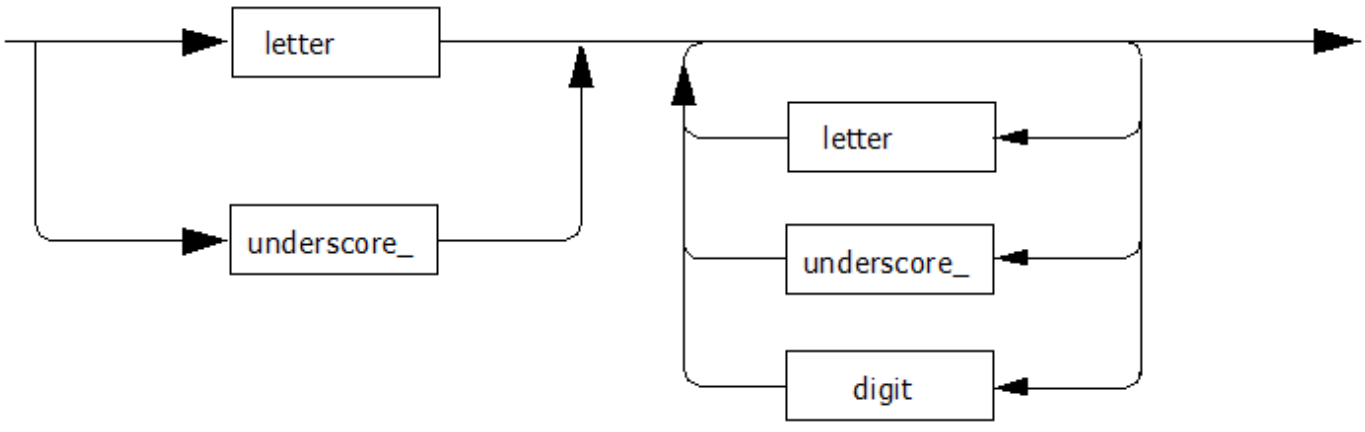
For names these restrictions apply:

- The name cannot start with a digit. It can start with a letter or with an underscore (_). This can be followed by letters, digits or the underscore (_), for example `MyStation`, `_MyStation`, `MyStation1`, `My_Station_1`.

  The name of an object cannot start with a digit, i.e., you cannot specify `1Station`. Special characters that have a meaning in *SimTalk*, such as +, -, *, /, =, <, >, etc., cannot be part of a name.
- Names of the **keywords** and names of the built-in functions and methods are not allowed.
- Names are **not case-sensitive**, *Plant Simulation* does not distinguish between `upper`, `UpPER`, and `UPPER`.

> **Note**
> Data that you enter is case-sensitive though. When comparing string values *Plant Simulation* takes upper- and lower-casing into account.

The syntax diagram looks like this:



Specify names identifying the role of an object or variable whenever this is possible. `forklift.destination := .building1.station_1` is more meaningful than `MU1.attrib := .Frame1.Station21`.

See also

Name [general description]

Predefined Names

Predefined Names

The predefined names *reset*, *init*, *autoexec*, and *endSim* perform their tasks during the respective phase of the simulation.

Remarks

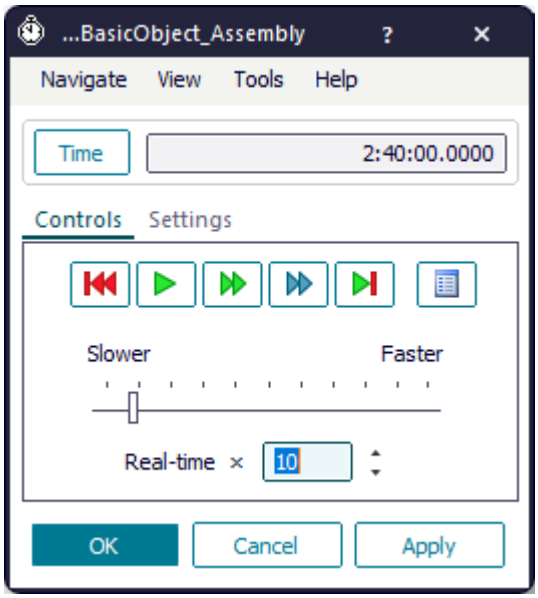
Before and after a simulation run *Plant Simulation* has to perform certain tasks: It has to initialize lists, analyze tables, and show the results of a simulation run on screen or write them to a file.

We defined specific method names to the four consecutive phases of a simulation:

- **reset** to reset the simulation model to its initial state
- **init** to initialize the simulation model
- **autoexec** to automatically start the simulation run
- **endSim** to terminate the simulation run

Plant Simulation calls each method in the respective phase.

These methods correspond to the buttons **Reset Simulation** and **Start/Stop Simulation** on the **Tab Controls** of the *EventController*.



autoexec [SimTalk]

Plant Simulation calls the method named *autoexec* immediately after opening the simulation model provided that the *autoexec method* is located in the *Class Library*.

Remarks

You can create and use any number of *autoexec* methods.

To make the *ExperimentManager* run models in a batch run, start the experiment in the experiment study in the *autoexec method* with the method *startExperiment*. Create the *autoexec method* in the *Class Library* and type in the following source code:

```
// the name of the method is autoexec
.Models.MySimulation.ExperimentManager.startExperiment
```

See also

ExperimentManager [object]

autoexecLoadObj [SimTalk]

Plant Simulation runs all loaded method classes named *autoexecLoadObj* when it loads an object file (*.psobj) or library file (*.pslib) into the simulation model.

Remarks

In addition, *Plant Simulation* executes all *user-defined attributes* of data type *method*, which are located in any of the loaded classes and which are named *autoexecLoadObj*.

Plant Simulation also executes *autoexecLoadObj* methods:

- When you load an object file with the method *loadObjectAs*.
- When you select the commands **Load Object**, **Load Object into Folder**, or **Update** .
- When you add a library in the dialog **Manage Class Library**.

Plant Simulation does not execute *autoexecLoadObj* methods when you open a model file.

Parameters

Optionally *Plant Simulation* can pass two parameters of data type *boolean* to this method.

- *Plant Simulation* sets the first parameter to true, when the *Class Library* is updated and to false, when loading an object.

  Note

  For *autoexecLoadObj* methods, which are part of a library, you should ignore the first parameter, as *Plant Simulation* determines according to the version number if the library will be updated or not when loading a library.

- If the method is located within a library, the second parameter tells if the library was updated (true), or if the library was added to the simulation model (false).

  The second optional parameter can also be of data type *string*. If a library is being updated, the old version number of the library will be passed to this parameter. Then the library is not being updated but added, and an empty string `""` will be passed to the parameter.

SimTalk

loadObjectAs [SimTalk]

See also

Load Object

Load Object into Folder

Update Class Library

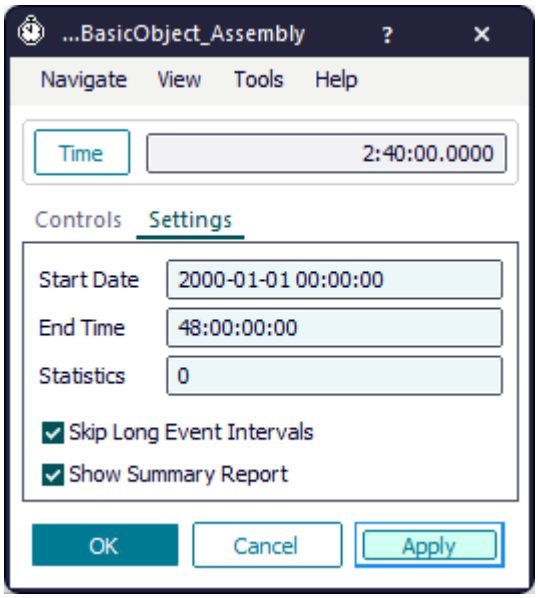
Manage Class Library

endSim [SimTalk]

Plant Simulation calls all methods named *endSim* at the end of a simulation run.

Remarks

In an *endSim* method you can, for example, evaluate the data of the simulation run and save it to a file. The simulation ends, when the *EventController* has processed all events in the **List of scheduled events** or when *Plant Simulation* reaches the **End Time**, which you typed into the text box **End Time** on the tab **Settings** in the *EventController*.



Compare the examples below to get an idea of what you can accomplish in an *endSim method*.

- To show the simulated **Availability** at the end of the simulation run in the *Console* in the model, create an *endSim method*, and type in the following source code:

```
print EventController.simTime," Simulated availability in the afternoon:
", round(100 * (1 - MyStation.statFailPortion),2)," %"
```

- To **close a database**, create an *endSim method*, and type in the following source code:

```
closeMyDatabase  // is the name of the method that closes the database
                 // closeMyDatabase can accomplish other functions as well
```

- To **write the default return value** of the contents list, namely the *array*, into a table with the method *copyToTable*, create an *endSim method*, and type in the following source code:

```
// Copies the contents lists into the table MyContentsList
// as an array with the method copyToTable
```

```
var x,y: real, row : integer
var a: any := Buffer.ContentsList
MyContentsList[1,1] := "Buffer"
a.copyToTable(MyContentsList,2,1)
row := MyContentsList.yDim + 2 // inserts an empty line
MyContentsList[1,row] := "Conveyor"
print a // prints the contents list as an array to the console
print "Number of items: ",a.yDim
var b := Conveyor.ContentsList // local variable is of data type any
print b
print "Number of items: ",b.yDim
b.copyToTable(MyContentsList,2,row)
MyContentsList.opendialog
```

```
// Writes the ContentsList into a table
// as previous versions of Plant Simulation did
Buffer.ContentsList(ContentsListBuffer)
Conveyor.ContentsList(ContentsListConveyor)
```

Call Sequence of the endSim Methods

Plant Simulation always executes the *endSim methods* of the objects in the reverse order in which you inserted them.

Within a *Frame Plant Simulation* always executes the *init* and the *reset methods* of the *Frame* itself after executing the *endSim methods* of all objects which you inserted into it.

Note

When you **Cut** (**Ctrl+X**) and **Paste** (**Ctrl+V**) an object, *Plant Simulation* cuts this object and inserts it anew thus changing the order in which the objects were inserted into the *Frame*.

Note

When you bring an object to the front or send it to the back in the *Frame* by clicking **Bring to Front** or **Send to Back** on the **Icons** ribbon tab of the *Frame*, this changes the order in which the objects were inserted into the *Frame*.

Note

Plant Simulation treats *endSim methods*, which you programmed in a *Method* object, and *endSim methods*, which you attached to an object as a *user-defined attribute* of data type *method*, the same.

SimTalk

copyToTable [SimTalk]

See also

[Write the Content List into a Table for Further Processing]

[End Time [EventController]]

[List of Events]

init

init [SimTalk] - predefined name

The **Init** event is the first event that is executed when you start the simulation of a simulation model

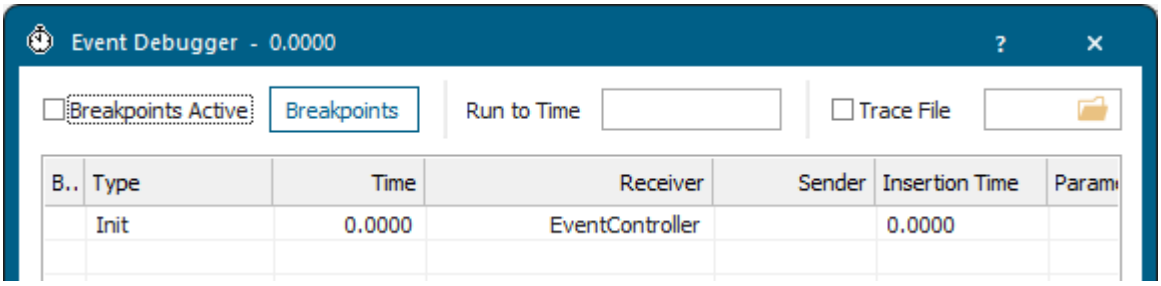
Remarks

Among others *Plant Simulation* executes all methods named *init* within the current simulation model.

Init controls are executed before the events are computed. The normal *init methods*, on the other hand, are executed after the initial events have been computed.

Example

Click  in the *EventController* to open the **List of scheduled events**. You'll see the first event to be processed is the **Init** event.



When you click  , *Plant Simulation* processes the event and proceeds to the next one.

Compare the examples below to get an idea of what you can accomplish in an *init method*.

- To start reducing the **Availability** of the machine after 12 hours have been simulated, create an *init method*, and type in the following source code.

```
&setFailure.methCall(str_to_time("12:0:0"))
```

- To **create and insert** a *Transporter* on a *Track*, use the *SimTalk* method *create*, create an *init method*, and type in the following source code:

```
.MUs.Transporter.create(Track)       // inserts a Transporter anywhere on
the Track
.MUs.Transporter.create(Track, 5.5)  // inserts a Transporter 5.5 meters
along the length of the Track
```

- To reopen the **Entrance** and the **Exit** of a station, create a *user-defined attribute* of data type *method* for this station and name it *init*. Then, type in the following source code:

```
self.~.EntranceLocked := false
self.~.ExitLocked := false
```

Parameters in Init Methods

You can also call *init methods* with a parameter of data type *integer*.

During the **init phase** the *init controls* are called first.

Then the *init methods* with parameters are called. This parameter has the value 1.

Then the objects are initialized.

After that the *init methods* with or without parameter are called. *Init methods* with a parameter are passed the value 2.

```
param phase: integer

if phase = 1
-- Objects are not yet initialized

-- The station has not yet computed the event for the first failure
-- so we can set the failure profile
  Station.Availability = 95;
   Station.MTTR = 10:00
else
-- Objects are already initialized

-- Now the first failure event is created based on
-- the parameters set in phase 1
    print Station.GetDisruptionBeginTime
end
```

See also

Init Control [general description]

Init Control [WorkerPool]

Init Control [EventController]

Init Control [Transporter]

List of Events

Init Methods and Init Controls

In certain cases it might be desirable that procedures are to be executed before a simulation run starts. You might, for example, set the number of *Workers* in a *WorkerPool* or to set general parameters of the objects.

Remarks

You can execute procedures before the simulation starts in *Plant Simulation* with:

- An *init control*: This is a built-in attribute of the material flow objects named *InitCtrl* that points to a *Method* object or a *user-defined attribute* of data type *method*.
- An *init method*: This is an object of type *Method* that you named `init`. It can also be a *user-defined attribute* of data type *method* that you attached to an object and named `init`. You can also configure the *init method* with a parameter of data type *integer*.

> **Note**
>
> You can define a parameter of data type *integer* for an *init method* to set two phases when they will be called.
>
> In phase 1 the *init method* is called before the objects have initialized themselves.
>
> In phase 2 the *init method* is called once the objects have initialized themselves.
>
> If you want to change the failure parameters, for example, you will to do this in phase 1 when the object has not yet calculated the time for the next failure.

Plant Simulation executes both in the same way, the only difference is the point in time at which they are being called.

- The *init controls* are executed **before** the objects are initialized.
- The *init methods* are executed **after** all objects have been initialized.

This difference is crucial. You can, for example, change the **Workers to Create Table** of the *WorkerPool* for the next simulation run with an *init control*. You cannot achieve this with an *init method*, as this method would be called too late.

The **call sequence** of the *init controls* and the *init methods* is as follows:

1. **Init Control** of the *EventController*
2. *Init controls* of all other objects, for example the **Init Control** of the *Transporter*, **Init Control** of the *WorkerPool*, including the *model frame*, etc.
3. *Init methods* with parameter. This parameter has the value 1 here.
4. Initialization of the objects
5. *Init methods* with or without parameter. *Init methods* with parameter are passed the value 2 here.

6. *Init methods* of the inserted objects
7. *Init method* of the *model frame*

SimTalk

InitCtrl [SimTalk] - general description

See also

Init Control [EventController]

Init Control [Transporter]

Init Control [WorkerPool]

Back to

Init Methods and Init Controls

Video on YouTube

https://youtu.be/5t-wLNmKpbU?si=4VbDSjs2-CSZXYU5&t=1246

Call Sequence of the Init Controls

Plant Simulation calls an *init control* once at the beginning of the simulation run during the initialization phase.

Remarks

Plant Simulation calls an *init control* before objects are initialized, and before *init methods* are executed as follows:

1. *Init control* of the *EventController*
2. *Init control* of all other objects

   If the object is a *Frame*, the *init controls* of the objects inserted into this *Frame* are called recursively
3. *Init control* of the *model frame*

The execution of the *init controls* always starts from the attribute *InitCtrl* of the *EventController* and then continues with objects and *Frames* in the *model frame*.

The *init controls* in the *model frame* are called in the sequence in which they have been inserted into the *model frame*. Given this, the *init controls* of all objects and *frames* are called in the reverse order in which they were inserted. Here the rule applies as well, that the *init control* of an object that is inserted into a *Frame* always is executed before the *init control* of the *Frame* in which it is inserted.

The last *init control* called always is the *init control* of the *model frame*. After this process is finished, all objects in the model are initialized, for example *Workers* are created, machines are set up, etc.

Back to

Init Methods and Init Controls

Call Sequence of the Init Methods

After the initialization of all objects in a *model frame* is finished, *Plant Simulation* executes the *init methods*.

Remarks

The sequence in which the *init methods* are executed is the same as the call sequence of the *init controls*. Within a *Frame*, *Plant Simulation* always executes the *init methods* of objects, which are inserted into the *Frame*, before executing the *init methods* of the *Frame* itself.

Back to

Init Methods and Init Controls

Example of the Call Sequence

The example illustrates the sequence in which *init controls* and *init methods* are called in more detail.

Remarks

For our example we inserted the objects into the *Frames* in the following sequence:

1. *EventController*
2. *Method object* named *Init* in the *Frame* named *Model*
3. *WorkerPool*
4. *Frame1*
5. *Frame2*
6. *Station2* in *Frame2*
7. *Station* in *Model*
8. *Station1* in *Frame1*
9. *Init method* in *Frame1*
10. *Init method* in *Frame2*

In the screenshot below we indicated the sequence in which the objects are inserted with the numbers above each object. All objects in the model, except for the objects of type *Method*, but including the *Frames*, have *init controls* attached to them.

The sequence in which the *init controls* are called is illustrated with the red number to the right of the object.

The first *init control* to be executed always is the *init control* attached to the *EventController*.

This is followed by the *init controls* of the objects in the *Frame* named *Model*. In the example we inserted the object *Station* as the last object into the *Frame* named *Model*, so that its *init control* is called first.

This is followed by the sub-frame *Frame2*. As it contains another object with an *init control*, namely *Station2*, the *init control* of *Station2* is executed before the *init control* of the sub-frame *Frame2*.

Then the *init control* of the first station, *Station1*, followed by the first *Frame*, *Frame1*, is called.

As we inserted the *WorkerPool* last, its *init control* is the last one to be called before the *init control* of the *Frame* named *Model* is called.

The last *init control* to be executed always is the *init control* of the *Frame* named *Model*.

After the *init controls* were executed, the *init methods* are executed in the same way as the *init controls* were. We illustrate this sequence with the red numbers to the right of the objects in the screenshot below.

After the *init control* of the *Frame* named *Model* was called as the eighth *init control*, the initialization of the objects in the *Frame* starts.

After the initialization, the *init method* of the object named *Station* is called as the ninth procedure, followed by the *init method* of *Station2* and the *init method* object inserted in *Frame2*.

After that the *init method* of *Station1* is called, followed by the *init method* object in *Frame1*.

Then the *init method* of the *WorkerPool* is called, followed by the *init method* object of the *Frame* named *Model*.

The example illustrates that the call procedure and the call sequence of the *init controls* and of the *init methods* is the same.

The *init controls* and the *init methods* are called recursively for all of their objects within a *Frame*. If the respective objects themselves are *Frames*, then the *init controls* and the *init methods* that are inserted into *Frames* are called recursively again. This procedure continues until all *init controls* or *init methods* are executed, except for those that belong to the *Frame* named *Model*.

Back to

Init Methods and Init Controls

Call Sequence of Reset and Init Methods

Plant Simulation always executes the *init methods* and the *reset methods* of the objects in the reverse order in which you inserted them.

Remarks

Within a *Frame Plant Simulation* always executes the *init methods* and the *reset methods* of the *Frame* itself after executing the *init methods* and *reset methods* of all object which you inserted into it.

Note

When you **Cut** (**Ctrl+X**) and **Paste** (**Ctrl+V**) an object, *Plant Simulation* cuts this object and inserts it anew thus changing the order in which the objects were inserted into the *Frame*.

Note

When you bring an object to the front or send it to the back in the *Frame* by clicking **Bring to Front** or **Send to Back** on the **Icons** ribbon tab of the *Frame*, this changes the order in which the objects were inserted into the *Frame*.

Note

Plant Simulation treats *init methods*, which you programmed in a *Method* object, and *init methods*, which you attached to an object as a *user-defined attribute* or type *method*, the same.

See also

reset [SimTalk] - predefined name

Back to

Init Methods and Init Controls

reset [SimTalk] - predefined name

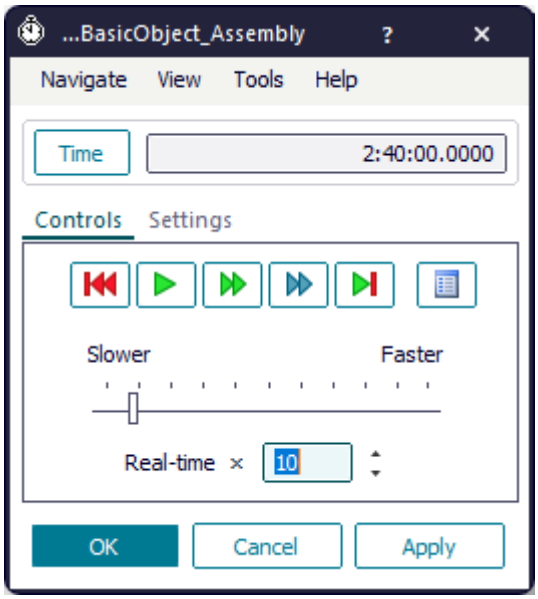
Plant Simulation calls all methods in your model named *reset* when you click **Reset Simulation** in the *EventController*.

Remarks

Plant Simulation also deletes all events from the **List of scheduled events**, sets the simulation time to 0, deletes statistics data, and clears all failures.

> **Note**
>
> The **Request Control**, the **Receive Control**, and the **Release Control** of the *Importer* are not being called when you reset the simulation.

> **Note**
>
> *executeIn* calls do not work in *reset methods*, as the *EventController* deletes every scheduled event during the *reset* phase. You can use an *init method* instead to schedule events.



Compare the examples below to get an idea of what you can accomplish in a *reset method*.

- To **delete all parts** in your simulation model, when you reset it, create a *reset method*, and type in this source code:

```
deleteMovables
```

- To reduce the **simulation speed** when you reset the model, create a *reset method*, and type in this source code:

```
EventController.Speed := 60
```

- To reset the **Availability** of the machine at the start of the next simulation run to one hundred percent, create a *reset method*, and type in this source code:

```
MyStation.Availability := 100
```

- To **delete the contents of a table** when you reset the model, create a *user-defined attribute* of data type *method* for this table and name it *reset*. Then type in this source code:

```
self.~.delete
```

- To **delete the contents** of the *DataTable* named *InventoryTable* and reset the counter of the *Variable* named *NextNumber* to 1, create a *reset method*, and type in this source code:

```
InventoryTable.delete({0,1}..{*,*})
NextNumber := 1
```

- To **delete the result numbers** from the object *Comment*, create a *reset method*, and type in this source code:

```
Comment.Text := "MU Type,
Lifetime"+strChr(13)+strChr(10)+"----------------------"
```

Call Sequence of the Reset and Init Methods

Plant Simulation always executes the *init methods* and *reset methods* of the objects in the reverse order in which you inserted them.

Within a *Frame Plant Simulation* always executes the *init methods* and *reset methods* of the *Frame* itself after executing the *init methods* and *reset methods* of all object which you inserted into it.

Note

When you **Cut** (**Ctrl+X**) and **Paste** (**Ctrl+V**) an object, *Plant Simulation* cuts this object and inserts it anew thus changing the order in which the objects were inserted into the *Frame*.

Note

When you bring an object to the front or send it to the back in the *Frame* by clicking **Bring to Front** or **Send to Back** on the **Icons** ribbon tab of the *Frame*, this changes the order in which the objects were inserted into the *Frame*.

Note

Plant Simulation treats *reset methods*, which you programmed in a *Method* object, and *reset methods*, which you attached to an object as a *user-defined attribute* or type *method* the same.

SimTalk

executeIn [SimTalk]

See also

reset [SimTalk] - EventController

init [SimTalk] - predefined name

Reset Simulation [EventController]

List of Events

Request Control [processing importer]

Receive Control [processing importer]

Release Control [processing importer]

Keywords

Keywords

A **keyword** is a reserved word in *SimTalk* that you cannot use as an identifier in your source code. You cannot use it to name an object, a local variable, or a function.

Remarks

SimTalk provides these **keywords**.

and [SimTalk]	exitloop [SimTalk] - keyword	print [SimTalk] - output function	true [SimTalk]
basis [SimTalk] - anonymous identifier	false [SimTalk]	prio [SimTalk]	until [SimTalk]
byref [SimTalk] - reference operator	for [SimTalk]	repeat [SimTalk]	var [SimTalk]
case [SimTalk]	forget [SimTalk]	result [SimTalk]	void [SimTalk] - variable

continue [SimTalk]	if [SimTalk]	return [SimTalk]	wait [SimTalk]
create [SimTalk] - local list	loop [SimTalk]	root [SimTalk] - anonymous identifier	waitExpired [SimTalk]
current [SimTalk] - anonymous identifier	mod [SimTalk]	rootfolder [SimTalk] - anonymous identifier	waituntil [SimTalk]
div [SimTalk]	next [SimTalk]	self [SimTalk] - anonymous identifier	when [SimTalk]
downto [SimTalk]	not [SimTalk]	stopuntil [SimTalk]	while [SimTalk]
else [SimTalk]	or [SimTalk]	switch [SimTalk]	
elseif [SimTalk]	param [SimTalk]	then [SimTalk]	
end [SimTalk]	pi [SimTalk] - value	to [SimTalk]	

Plant Simulation shows **keywords** in the source code in this color `false`.

Keywords are not case-sensitive **IntEgeR** and **inTEGer** are considered to be the same. When you add letters or numbers to the beginning or the end of a keyword, *Plant Simulation* does not treat it as a keyword any more. The string **list** as a name for a *user-defined method* is not allowed, while `alist` or `list0` are allowed.

Note

The function *checkID* checks if you can use an expression as the name of an object. The function considers all keywords and function calls.

All **data types** are keywords as well.

Acceleration [SimTalk] - data type	JSON [SimTalk] - data type	Real [SimTalk] - data type
Any [SimTalk] - data type	Length [SimTalk] - data type	Speed [SimTalk] - data type
Array [SimTalk] - data type	List [SimTalk] - data type	Stack [SimTalk] - data type
Boolean [SimTalk] - data type	Method [SimTalk] - data type	String [SimTalk] - data type
Date [SimTalk] - data type	Data Types [SimTalk]	Table [SimTalk] - data type
DateTime [SimTalk] - data type	Object [SimTalk] - data type	Time [SimTalk] - data type
Integer [SimTalk] - data type	Queue [SimTalk] - data type	Weight [SimTalk] - data type

See also

Colors for Syntax Highlighting

Data Types [SimTalk]

checkID [SimTalk]

and [SimTalk]

The keyword **and** designates the logical operator AND.

See also

The Logical Operators AND and OR

case [SimTalk]

The keyword **case** is part of the control structure *switch*, together with the keyword *switch*.

See also

Check for a Large Amount of Different Values with switch [SimTalk]

switch [SimTalk]

continue [SimTalk]

The keyword **continue** skips the rest of the loop iteration and continues with the next iteration.

Remarks

If the current iteration was the last iteration, **continue** exits the loop.

If you want to continue with the next iteration for an outer loop, specify an *integer* value with the number of loops for which **continue** is to apply after the keyword **continue**.

If you type in `continue 2` for example, *Plant Simulation* exits the inner loop and executes the next iteration of the outer loop.

See also

Loops [SimTalk]

div [SimTalk]

The keyword **div** represents an **integer division**.

See also

Arithmetic Operators

downto [SimTalk]

The keyword **downto** is part of the *for-loop*, together with the keywords *for*, *next*, and *to*.

See also

for loop [SimTalk]

for [SimTalk], keyword

next [SimTalk], keyword

to [SimTalk], keyword

else [SimTalk]

The keyword **else** is part of the `if-else-end-elseif`-statement.

See also

Branching with if-else-end

Multiple Branching with if-elseif-end

elseif [SimTalk]

The keyword **elseif** is part of the `if-else-end-elseif`-statement

See also

Branching with if-else-end

Multiple Branching with if-elseif-end

end [SimTalk]

The keyword **end** is part of branching with *if-else* and of the *while loop*.

Remarks

Indent the instructions within branches or a loop by at least one space. This enables you to recognize if the instruction is completed correctly.

See also

Branching with if-else-end

while loop [SimTalk]

exitloop [SimTalk] - keyword

The keyword **exitloop** exits any of the *loops*, which *Plant Simulation* provides.

Remarks

You can also set the number of loops to be exited by entering an *integer* number, for example `exitloop 2 // two loops`.

See also

Exit a Loop with exitLoop

Loops [SimTalk]

false [SimTalk]

The keyword **false** designates the boolean operator false.

See also

true [SimTalk]

for [SimTalk]

The keyword **for** is part of the *for-loop*.

See also

for loop [SimTalk]

forget [SimTalk]

The keyword **forget** applies to local variables of data types *table*, *list*, *queue*, and *stack* as well as to the data type *any*.

Remarks

The keyword **forget** does the following:

- If you apply the keyword **forget** to one of the *list* data types, it deletes the local list you created with the method *create*.

  Normally you will not use *forget* because *Plant Simulation* deletes the list anyway when the *Method* is finished.

  You will use it to execute the statement *create* again.

  Example

```
var t: table
t.create
t.forget
t.create
```

- If you apply the keyword **forget** to a local variable of data type *any*, it resets this variable, so that you can assign another data type afterward.

  Example

```
var a: any
a := "Test"
forget a
a := 123
```

See also

create [SimTalk] - local list

if [SimTalk]

The keyword **if** is part of the `if-else-end-elseif`-statement.

See also

Branching with if-else-end

Multiple Branching with if-elseif-end

loop [SimTalk]

The keyword **loop** is part of the *loops* which *Plant Simulation* provides.

Remarks

You can use the keyword **loop** to write loops into a single line of code instead of into several lines.

`while x < 5 x += 1 end`	`while x < 5 loop x += 1 end`
`for var i := 1 to 5 print i next`	`for var i := 1 to 5 loop print i next`

See also

Loops [SimTalk]

mod [SimTalk]

The keyword **mod** represents the **integer modulo operation**.

Example

```
Variable := Variable mod 360
```

See also

Arithmetic Operators

next [SimTalk]

The keyword **next** is part of the *for-loop*.

See also

for loop [SimTalk]

not [SimTalk]

The keyword **not** designates the *logical operator NOT*.

See also

The Logical Operator NOT

or [SimTalk]

The keyword **or** designates the *logical operators AND and OR*.

See also

The Logical Operators AND and OR

param [SimTalk]

The keyword **param** declares *parameters* in the source code of a *Method*.

Examples

```
param v1,v2: integer, name: string
```

```
param maxValue:real := 1.0 -> real // optional parameter
return z_uniform(0,maxValue)
```

->, <data type of the return value>

The expression **->** (hyphen plus right angle bracket) declares the *data type of the return value* of the *Method*.

```
-> boolean // the data type of the return value is boolean
```

or

```
param v1,v2: integer, name: string -> boolean
```

See also

Parameters [SimTalk]

Declare Parameters

Declare the Function Result of a Method

Programming a Method

prio [SimTalk]

The keyword **prio** sets the priority of execution for the instructions in a *waituntil statement* or a *stopuntil statement*.

See also

waituntil [SimTalk] and stopuntil [SimTalk]

repeat [SimTalk]

The keyword **repeat** is part of the *repeat loop*.

See also

repeat loop [SimTalk]

result [SimTalk]

The keyword **result** designates the *function result of a Method*.

See also

Declare the Function Result of a Method

return [SimTalk]

The keyword **return** exits a method.

See also

Exiting Methods with the Keyword return

stopuntil [SimTalk]

The keyword **stopuntil** is part of the *stopuntil statement*.

Remarks

The *stopuntil-statement* suspends *Method execution* until the condition set in the statement evaluates to *true*.

See also

SimTime [SimTalk]

waituntil [SimTalk] and stopuntil [SimTalk]

Simultaneously Waking up Several Methods with stopuntil

Simultaneously Waking up Several Methods with waituntil

Simultaneously Waking up Several Methods

switch [SimTalk]

The keyword **switch** is part of the control structure *switch*, together with the keyword *case*.

See also

Check for a Large Amount of Different Values with switch [SimTalk]

case [SimTalk]

then [SimTalk]

The keyword **then** is part of the `if-else-end-elseif`-statement.

Remarks

Use the keyword **then** to write loops into a single line of code instead of into several lines.

`if x = 0 y := 1 end`	`if x = 0 then y := 1 end`

See also

Branching with if-else-end

Multiple Branching with if-elseif-end

to [SimTalk]

The keyword **to** is part of the *for-loop*.

See also

for loop [SimTalk]

true [SimTalk]

The keyword **true** designates the boolean operator true.

See also

false [SimTalk]

until [SimTalk]

The keyword **until** is part of the *repeat-loop* and the *from-until-loop*.

See also

repeat loop [SimTalk]

var [SimTalk]

The keyword **var** declares a *local variable* in the *Method*.

Remarks

A variable must have the same data type as the value you want to assign to it.

```
var myVariable := 42
```

If the initial value does not provide enough information, or if there is no initial value, specify the data type by typing it int after the variable, separated by a colon:

```
var myDouble: real := 3.1415
```

You can create *arrays* using brackets `[]` and access their elements by typing the index within the brackets.

```
var myIntegerArray: integer[]
myIntegerArray.append(3)
print myIntegerArray[1]
```

See also

Declare Local Variables in the Source Code

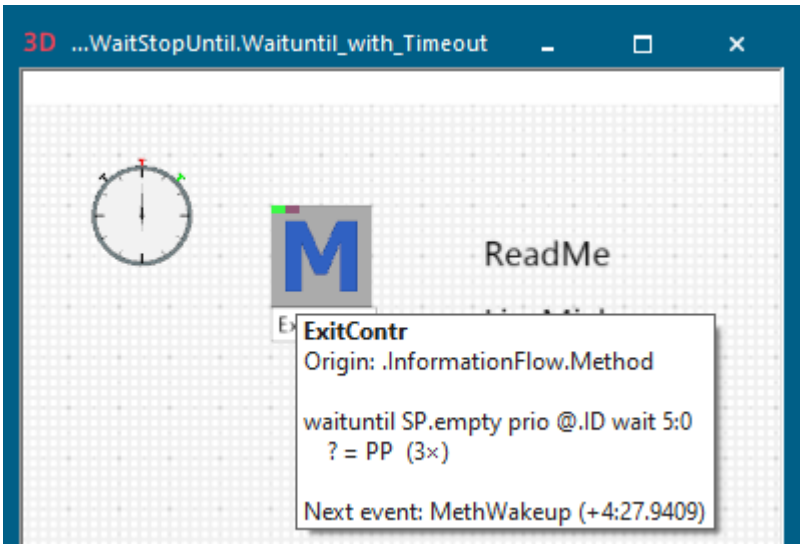
wait [SimTalk]

The keyword **wait** interrupts the execution of the *Method* for the specified amount of simulation time. The execution of the *Method* continues as soon as the time of the *EventController* advanced by the amount of time you specified for the *wait-statement*.

Remarks

If a control is being executed during the **reset phase** and suspends itself with a **wait instruction**, *Plant Simulation* deletes the suspension at the end of the **reset phase**.

A *Method* that is suspended by a **wait instruction** shows this in the *tooltip* in the *Frame window*.




```
@.move  -- the part exits
?.ExitLocked := true
wait 5  -- the exit is blocked for 5 seconds
?.ExitLocked := false
```

See also

Interrupting Method Execution with Wait

sleep [SimTalk], keyword

Time Limitation for Waituntil and Stopuntil

Suspending Methods

Video on YouTube

https://youtu.be/5t-wLNmKpbU?si=B7E1bFnWDboTPb7t&t=677

waitExpired [SimTalk]

The keyword **waitExpired** is part of the *waituntil statement* and the *stopuntil statement*.

Remarks

waitExpired sets a time limit for the *waituntil/stopuntil statement*.

If the *waituntil/stopuntil statement* has been suspended for this duration, *Plant Simumlation* will then wake it up, although the condition is not met yet.

- *Plant Simulation* sets the keyword `waitExpired` to *true* if the statement wakes up due to the time limit.
- If the suspension is terminated due to the condition, *Plant Simulation* sets **waitExpired** to *false*.

Examples

```
waituntil Station.Empty wait 60
   if waitExpired
       Station.deleteMovables
   else
       .MUs.Part.create(Station)
   end
```

```
var t:time := z_uniform(1, 50:0, 70:0)
stopuntil GateOpen prio @.ID wait t

if waitExpired
    @.name := "Expired"
else
    @.name := "Unexpired"
end

@.move(PP)
GateOpen := false
```

```
waituntil Station.empty prio @.ID wait 5:0

if waitExpired
    @.name := "Expired"
else
    @.name := "Unexpired"
end

if NOT @.umlagern(Station)
    print Eventcontroller.simTime," The waiuntil expired and the MU
",@.ID," was added into the forward blocking list."
end
```

See also

The waituntil statement and the stopuntil statement

waituntil [SimTalk]

The keyword **waituntil** is part of the *waituntil statement*.

Remarks

The *waituntil-statement* suspends *Method execution* until the condition set in the statement evaluates to *true*.

See also

SimTime [SimTalk]

waituntil [SimTalk] and stopuntil [SimTalk]

Simultaneously Waking up Several Methods with stopuntil

Simultaneously Waking up Several Methods with waituntil

Simultaneously Waking up Several Methods

when [SimTalk]

The keyword **when** is part of the *when-then-else statement*. It also is part of the control structure *switch*.

See also

Conditional Expressions with when-then-else

switch, control structure

while [SimTalk]

The keyword **while** is part of the *while-loop*.

See also

The while-loop

Anonymous Identifiers

Anonymous Identifiers [described]

Anonymous identifiers make using a *Method* more flexible and independent of its context.

Remarks

Anonymous identifiers are, for example **@**, **basis**, **current**, **?**, **root**, **RootFolder**, and **self**.

You can insert an anonymous identifier into different models without having to modify the source code of the *Method* that contains it.

Normally, the name of a control you programmed in a *Method* does not change in your model. The path to the *Method* changes from model to model, though. If a control requires the current path, you can query it by using **anonymous identifiers**.

See also

Location [SimTalk] - material flow objects

Video on YouTube

https://youtu.be/pbOQD0p4hO4?si=dZeZUnfEYpNkq8uQ&t=552

@ [SimTalk] - anonymous identifier

The anonymous identifier **@** (**at sign**) designates the *MU* that triggered the respective control.

Remarks

If you specify an **Entrance Control** or an **Exit Control** in a material flow object, the anonymous identifier **@** enables you to access the *MU* that entered the object or is ready to exit the object.

The assignment is unique even if several *control methods* are triggered at the same simulation time, as *Plant Simulation* always processes the triggering *MU* completely before proceeding to the next *MU*.

Example

```
@.move(ParallelStation.succ(3))
```

? [SimTalk] - anonymous identifier

The anonymous identifier **?** (**question mark**) designates the material flow object or the control (*Method*) that called the *Method*.

Remarks

The anonymous identifier **?** enables a control to be used without modifications by several objects.

Example

```
?.Cont.move(E2) // moves the contents of the object that
                // called the method to the object E2
```

basis [SimTalk] - anonymous identifier

The anonymous identifier **basis** designates the *Class Library*.

Remarks

You can only use **basis** in comparisons, i.e., with **=** or **/=**.

Example

```
if location = basis
// in the class library
else
// inserted into a Frame
end
```

See also

Class Library

Relational Operators

current [SimTalk] - anonymous identifier

The anonymous identifier **current** returns the *Frame* within which the *Method* object is located.

Remarks

This way you can easily enter the location of a *Frame* into lists and tables or enter it as a parameter to methods in other *Frames*.

You can also use **current** to distinguish between a local and a global variable with the same name.

The parameter **name** identifies a local variable, while **current.name** designates the global variable.

Examples

```
print "Current pause ", current.pause
MyStation.pause := current.pause
var shift := root.ShiftCalendar.GetCurrShift
print "Current shift: ", shift

if not current.unplanned

if current.pause
   current.currIcon := "pause"
else
   current.currIcon := "working"
end
```

```
current.extendPath("Station")
-- checks if the object 'Station' exists in the current Frame
```

root [SimTalk] - anonymous identifier

The anonymous identifier **root** designates the topmost *Frame* in the hierarchy of *Frames*.

Remarks

From the topmost *Frame* on you can then navigate downward through the hierarchy of *Frames* in your model.

The identifier **root** is especially helpful if you do not know the name of the root *Frame*, i.e., the *Frame* that is located at the top of the hierarchy.

Example

```
Method1                   // same namespace
root.Frame2.Method1       // path and name
&Method1.executeIn(8.5)   // pass to EventController
Variable.execute          // call the referenced Method, note that the
                          // object Variable has the data type object
```

rootfolder [SimTalk] - anonymous identifier

The anonymous identifier **rootfolder** designates the folder in the *Class Library* in which you store *Methods* which a number of objects use.

Remarks

Using the root folder prevents wasting main memory and prevents long paths as *Plant Simulation* then calls this method once only from the *Class Library* instead of a large number of instances of this method located in different folders.

If you use the anonymous identifier `rootfolder` in a *Method*, the **rootfolder** designates a folder in the *Class Library* for which you have set the attribute *RootFolder*  . In this case *Plant Simulation* searches for such a folder, starting with the class of the *Method*, upward in the hierarchy of objects.

You can also use the anonymous identifier **rootfolder** within *control methods*. Here *Plant Simulation* looks for the folder for which you set the attribute *RootFolder* to `true`, starting with the class of the *Frame* into which you inserted the object in which you programmed the control.

Example

```
.UserObjects.rootfolder := true
```

See also

RootFolder [SimTalk] - attribute

Set the Root Folder for Your Simulation Model

self [SimTalk] - anonymous identifier

The anonymous identifier **self** designates the currently executed *Method*.

Remarks

Use **self** if it is likely that methods are renamed later on. Otherwise you may have to change each instance of the previous name to the new name.

```
self.executeIn(60)
self      // returns the path to and the name of the Method
self.Name // returns the name of the Method only
```

In a *user-defined attribute* of data type *method* the identifier *self* is the user-defined method attribute itself. Use *self.~* to access the object for which you created the *user-defined attribute*.

Return Value

The return value has the data type *object*.

Example

```
self.~.pause := true  // pauses the object for which you defined
                      // the user-defined attribute
```

Paths in SimTalk

Object Paths in SimTalk

If different objects are not located within the same *Frame*, i.e., within the same namespace, you have to add their path to the objects to uniquely identify them. Only then you can access them in a *Method*.

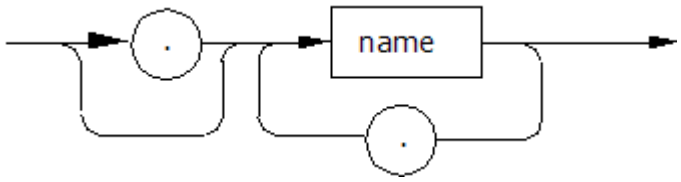
Remarks

An object path consists of a sequence of names separated by a period:

```
[.]name[.name.name[...]]
```

> **Note**
> Brackets designate optional components or parameters, meaning that you can, but do not have to specify them.

The syntax diagram of a path looks like this:



The **Syntax** of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

  > **Note**
  >
  > Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

  Optional parameters are listed within brackets. The expression *[, Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

  If a parameter has a default value, the signature shows the default value after the parameter

  If the *method* has a return value, the signature shows its data type after the arrow →.

Plant Simulation distinguishes between two kinds of paths.

- The **absolute path** starts with a period—standing for the **Class Library**—followed by the name of the folder and top-level *Frame*. Then a period and a name alternate until you reach the desired object, which again is preceded by a period.

  An *absolute path* might look like this: `.Models.MyPlant.Engine_Assembly_Anytown.MyStation`
- The **relative path** starts in the current namespace and identifies an object by a combination of built-in methods and names.

  In most cases the *relative path* starts with a *Frame*.

  A *relative path* might look like this: `MyPlant.Engine_Assembly_Anytown.MyStation`
- In addition, you can access a variable of data type *Object* using an **object reference**.

See also

The Absolute Path

The Absolute Path

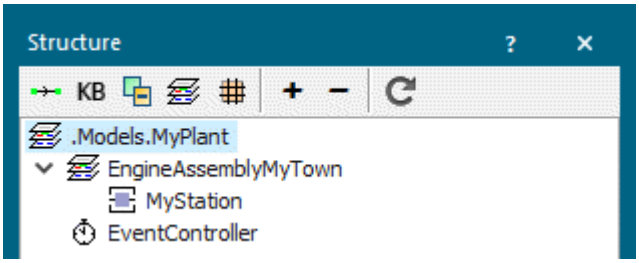
The *absolute path* starts with a period—standing for the *Class Library*—followed by the name of the folder and top-level *Frame*. Then a period and a name alternate until you reach the desired object, which again is preceded by a period.

Remarks

In the example below the *absolute path* of the *Station* named *MyStation* looks like this:

```
.Models.MyPlant.EngineAssemblyMyTown.MyStation
```

The structure of the model looks like this:



You might, for example, use the *absolute path* for a *control method*. This method might always execute certain actions before starting a simulation run, which never change. Then it makes sense to add this method to the *Class Library* and reference it using its absolute path that never changes.

You have to use the *absolute path* for *MUs* when you model a complex station, for example a receiving station of a plant with one or more *Sources*.

If, on the other hand, you want to use the same method, but with different source code, in instantiated *Frames*, you will use the **relative path**, to make sure that each *Frame* finds the method where you programmed actions specific to this *Frame*.

To insert the *absolute path* of the object into a text box with drag-and-drop, hold down the **Shift** and **Ctrl** keys.

See also

Class Library

The Relative Path

Object Reference

Change Values by Assigning a Value in SimTalk

Video on YouTube

https://youtu.be/MXDcx2yplxc?si=BvFouhsi3hu-7VdZ&t=771

The Relative Path

The *relative path* starts in the current namespace—formed by all objects located within a single *Frame*—or the *Frame* in which the *Method* object is located.

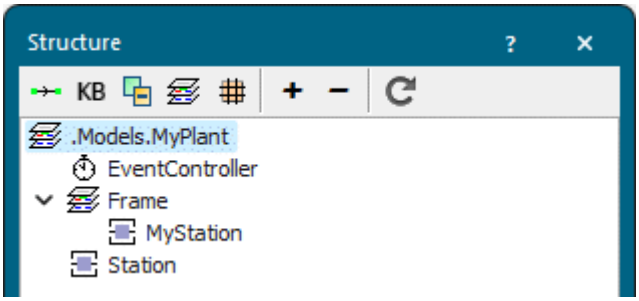
Remarks

The *relative path* starts with an anonymous identifier, a name in the current namespace, or a built-in method. Then a period follows and the name of a *Frame* or a period and a built-in method. The last entry is a period and the name of the object or a period and a built-in method or a period and an attribute.

In the example below the *relative path* of the *Station* which we inserted into a *Frame* named *MyPlant* looks like this:

```
MyPlant.EngineAssemblyMyTown.MyStation
```

The structure of the model looks like this:



This table lists where the relative path starts for the different objects:

Object	Relative path starts in
Built-in or *user-defined attribute* of data type *object* in a material flow object	the *Frame* which contains the material flow object.
Built-in or *user-defined attribute* of data type *object* in a *MU*	the parent *Frame* of the material flow object which contains the *MU*.
Built-in or *user-defined attribute* of data type *object* in a *Frame*	the *Frame* itself.
Source code of a *Method* or of a *user-defined attribute* of data type *method*	the *Frame* which contains the *Method* or the object which has the attribute.
DataTable or *user-defined attribute* of data type *table*, with columns of data type *object*	the *Frame* which contains the *DataTable* or in the parent object which has the attribute.

Object	Relative path starts in
User-defined attribute of data type *table* of a *MU*, with columns of data type *object*	the *Frame* in which the *MU* is located.
WorkerPool	the *Frame* in which the *Worker* is located.

Suppose that you want to access the object named *Station1* in a *Method*, which you inserted into a *Frame*. *Station1* is located in a sub-Frame named *Frame1*. The *relative path* then looks like this: `Frame1.Station1`.

Suppose that you are located in a *Method*, which is located in a sub-Frame of *Frame2*. Then you would like to access an object named *Station2* that is located in *Frame2*. Here the *relative path* looks like this: `Location.Station2` or `~.Station2`. The tilde ~ in the second case is an abbreviation of the built-in attribute *Location*.

You can change the starting position of the *relative path* with the keywords *root*, *self*, and *RootFolder*.

When dragging and dropping an object onto a text box, *Plant Simulation* inserts the relative path by default.

See also

The Absolute Path

Object Reference

Change Values by Assigning a Value in SimTalk

root [SimTalk] - anonymous identifier

self [SimTalk] - anonymous identifier

rootfolder [SimTalk] - anonymous identifier

Location [SimTalk] - material flow objects

Video on YouTube

https://youtu.be/MXDcx2yplxc?si=Izf_r6_u3TR3KKJp&t=600

Object Reference

A variable of data type *object* can accept a *relative* or an *absolute path* as well as an **object reference**. This also applies to global variables, local variables, formal parameters, and values in tables which have the data type *object*. An **object reference** always references a single object.

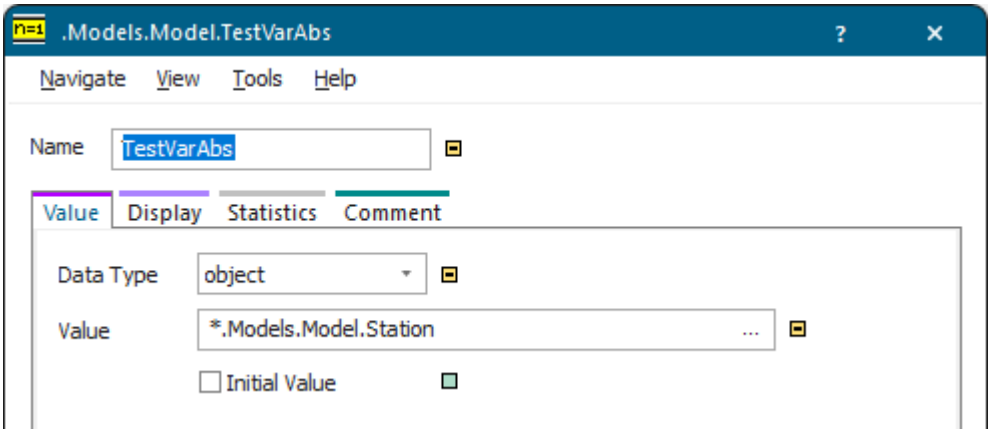
Remarks

Plant Simulation even points to the object when you rename it. If you delete the object, *Plant Simulation* cannot resolve the object reference anymore and does not find the object, even if you insert an object with the same name as the deleted object.

In the example below the object reference to a *Station* in the *Variable* looks like this:

```
*.Models.Model.Station
```

The asterisk ***** in front of the path denotes the object reference as such.



Note

We advise caution when using an **object reference**.

As a rule you can always use an **object reference** instead of an **absolute path**. If you need a relative addressing, use a **relative path** instead.

The main advantages of an **object reference** as compared to a path are a higher access speed and its tolerance toward renaming objects.

See also

Object [SimTalk] - data type

The Absolute Path

The Relative Path

Change Values by Assigning a Value in SimTalk

Video on YouTube

https://youtu.be/BvHLQCIjGIA?si=8T5fvvxRj9YxRz_2&t=118